



**Department of Electrical,
Computer, & Biomedical Engineering**
Faculty of Engineering
& Architectural Science

Course Title:	Embedded Systems Design
Course Number:	COE718
Semester/Year (e.g.F2016)	F2025

Instructor:	Dr. Gul Khan
-------------	--------------

Assignment/Lab Number:	Project Final Report
Assignment/Lab Title:	Media Center

Submission Date	Friday, November 28, 2025
Due Date	Friday, November 28, 2025

Student LAST Name	Student FIRST Name	Student Number	Section	Signature*
Aboud	Abdallah	501109619	2	

*By signing above you attest that you have contributed to this written lab report and confirm that all work you have contributed to this lab report is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a “0” on the work, an “F” in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: <http://www.torontomu.ca/content/dam/senate/policies/pol60.pdf>

I. ABSTRACT

This project implements an interactive media center on the Keil MCB1700 development board using the ARM Cortex-M3 microcontroller, the on-board LCD, joystick, speaker, and USB interface. The application integrates three major modules, a photo gallery that displays three custom character images stored as RGB565 C arrays, an MP3 audio player that streams audio from a host PC over USB while using the on-board potentiometer for volume control, and a game menu that hosts two original joystick controlled games. A state machine based main controller coordinates these modules, providing a single graphical menu and seamless switching between subapplications. The system was developed in C using the course support libraries as a foundation and then extended with new images, game logic, and user interface features to form a cohesive media application. Experimental testing on the physical board verified correct menu navigation, image display, and responsive game interaction, while revealing intermittent issues and inconsistencies with the USB audio playback. Overall, the project demonstrates practical understanding of embedded systems design, peripheral interfacing, and modular software architecture.

II. INTRODUCTION

Embedded systems are specialized computing platforms that perform dedicated functions while interacting with sensors, actuators, and users in real time. They appear in consumer electronics, automotive systems, and industrial controllers, where

reliable operation and efficient use of limited hardware resources are critical. In COE718, the Keil MCB1700 board and ARM Cortex-M3 microcontroller are used to introduce these concepts through hands-on experiments with LCD graphics, joystick input, USB communication, timers, and analog peripherals.

The final project extends these lab exercises by requiring the design of a complete media center application on the MCB1700. According to the project specification, the system must present a graphical main menu on the LCD, navigated with the joystick, and provide at least three core modules, a photo gallery that displays custom images, an audio player that streams sound from a PC via USB, and a game section containing at least one joystick controlled game. The goal is to integrate, adapt, and learn from all the concepts learned so far and apply it together to create a unified, user-friendly application.

In this implementation, the media center is organized around a top-level state machine that manages transitions between the main menu, photo gallery, audio player, and game menu. Each feature is implemented as a separate C module with clearly defined interfaces, which improves readability, debugging, and future expansion. The remainder of this report summarizes relevant background and lab material, the methodology and design behind the project, the architecture and implementation, testing outcomes, and lastly the main achievements, limitations, and possible enhancements to this project.

III. REVIEW

The media center project builds directly on the theoretical and practical material covered in COE718. Lecture notes on ARM Cortex-M3 programming, interrupt handling, and peripheral interfacing shaped the overall structure of the software and motivated the use of a clear, modular design. Concepts such as GPIO control, timer configuration, ADC/DAC usage, and simple real-time behaviour are all reused in the final system.

Laboratory exercises provided the main building blocks for this application. Early labs introduced the MCB1700 board, basic GLCD initialization, and simple text and graphics routines, which guided the implementation of the main menu and photo gallery screens. Later labs focused on joystick input and LED control, establishing patterns for reading button values, debouncing inputs, and providing visual feedback. These techniques are applied throughout the menu navigation, gallery browsing, and game interaction.

USB audio and timer-based peripherals were first explored through a reference USB audio example, which demonstrated how to configure the USB stack, handle audio transfers, and generate an output stream using timer interrupts and the DAC. The MP3 player module in the media center adapts this structure, integrating volume control via the on-board potentiometer and essentially linking the audio path to the project's state machine. The interim report and project manual

further influenced the final design by encouraging a modular breakdown into a main controller, separate feature modules, and a clear state/flow diagram, as shown in Figure 1, governing control flow.

IV. METHODOLOGY

The development of the media center began with interpreting the project specification and identifying all mandatory and optional features. Requirements were divided into three core functional modules being the photo gallery, audio player, and game menu, plus a shared graphical main menu and common services such as joystick input handling and application state control. A high-level block diagram and state machine flowchart were created to define how control would move between the main menu and each subapplication, and to clarify the responsibilities and interfaces of each module before implementation.

The software environment was then configured in Keil uVision using the recommended default startup and driver files for the LPC1768 microcontroller. A modular project structure was created with separate C source files for the main controller and menus (Blinky.c), the photo gallery (gallery.c), the game menu (game_menu.c), and each game (grid_hunter.c and highway_dodge.c), along with the audio player module. Initial placeholder functions were added so that the project compiled early in the process, allowing incremental development. Existing laboratory code and example projects for GLCD initialization, joystick reading, and USB audio streaming were reviewed and selectively adapted into

these modules, with care taken to keep interfaces consistent with the global state machine.

Implementation proceeded feature by feature. The main menu and joystick navigation were completed first to provide a stable entry point into the rest of the system. The photo gallery was then developed by designing three character images of three iconic videogame characters (Sonic, Super Mario, and Steve) and converting them from standard image formats into RGB565 C arrays using the GIMP application and following the instructions listed in the manual. The audio player module was integrated next by adapting the USB audio example of the USB stack, Timer0 interrupt, and DAC output were configured, and ADC readings from the on-board potentiometer were mapped to a software volume control. Finally, the game menu and the two games, Grid Hunter and Highway Dodge, were implemented as independent modules with their own drawing, input, and scoring logic.

Testing and refinement were carried out continuously. Each module was first verified in isolation using simple scenarios, for example, scrolling through gallery images, running Grid Hunter for a fixed number of rounds, or playing one Highway Dodge session to confirm collision detection. The integrated system was then exercised on the physical MCB1700 board using informal test cases covering menu transitions, response to joystick input, correct image sequencing, and proper return from each feature to the Main Menu. Debugging relied on LCD text prompts,

visual feedback on the screen, and uVision itself to observe variables such as the current state, joystick value, and game score. This iterative approach ensured that issues were detected early and that the final media center operated as a cohesive embedded application.

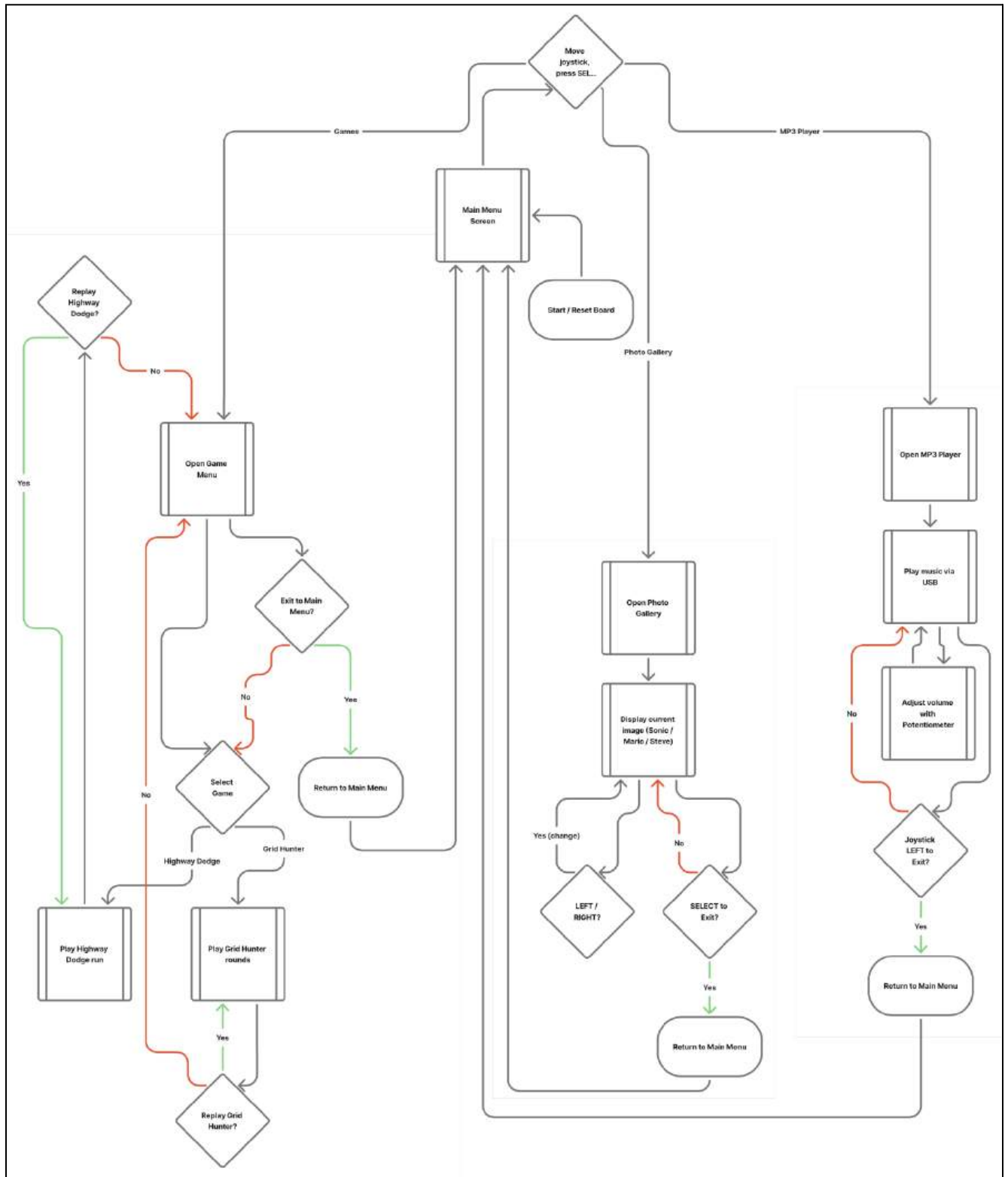


Figure 1: State/Control flow diagram of the Media Center

V. DESIGN

A. System Architecture

At a high level, the media center is organized as a set of subapplications managed by a global state machine. The central control logic resides in `Blinky.c` which provides the main function and the main menu implementation. When the board is powered on, the main function initializes the LEDs, GLCD, and joystick, and then repeatedly calls `main_menu()` inside an infinite loop. `main_menu()` is responsible for drawing the top-level media center menu and dispatching control to the photo gallery, audio player, or game menu based on joystick selection. Each subapplication runs to completion and then returns control to `main_menu()`, effectively using the menu as the project's home screen.

B. Application Menu Flow

The main menu is implemented in `Blinky.c`. It uses a variable `sel_num` to track the current selection among three items which are the photo gallery, audio, and game selection. The LCD background is set to purple, with white text for titles and instructions. The menu header and usage hints ("UP/DOWN - Scroll", "SELECT - Select") are printed on lines 0-3, while lines 5-7 are reserved for the three selectable options.

Within an infinite loop, `main_menu()` reads joystick movements using `get_button()`. Pressing UP or DOWN adjusts `sel_num` in the range 0-2, if the value moves outside this

range, it is wrapped back to 0. For each value of `sel_num`, the corresponding menu line is drawn with a light gray background to indicate the current selection, while the other two options are drawn on the purple background. When SELECT is pressed:

- If `sel_num == 0`, the Photo Gallery (`gallery()`) is invoked.
- If `sel_num == 1`, the Audio Player (`mp3_player()`) is invoked.
- If `sel_num == 2`, the Game Menu (`restart_game_menu()`) is invoked.

After the chosen module returns, `main_menu()` exits and control goes back to `main()`, which immediately calls it again, effectively re-drawing the top-level menu.



Figure 2: The Main Menu

C. Photo Gallery and Image Preparation

The photo gallery is implemented in `gallery.c`. Its purpose is to display three custom images of popular game characters: Sonic, Super Mario, and Steve. These images are stored in arrays

SONIC_pixel_data,
SUPER_MARIO_pixel_data, and
STEVE_pixel_data, which are declared as
extern in gallery.c and defined in separate C
files generated from GIMP.

To integrate the images into the embedded
system, the following process was followed
for each picture. The original PNG/JPEG
image was imported into GIMP. The image
was cropped and scaled to 100 x 125 pixels
so it would fit comfortably within the 320 x
240 LCD resolution while leaving space for
text on the screen. The image was converted
to an RGB format compatible with the
GLCD driver (RGB565). Using GIMP's
Export feature, the image was exported as C
source code, selecting the 16-bit 5-6-5
format option. The resulting .c file defined a
global pixel data array (for example,
SONIC_pixel_data[]), which was added to
the uVision project and included via an
extern declaration in gallery.c.

When gallery() is called, it clears the LCD,
sets a purple background, and prints a
header ("Photo Gallery") along with
instructions, "LEFT/RIGHT - Scroll" and
"SELECT - Exit." A variable sel_num
tracks which image is currently selected
(0-2). Inside the main loop, joystick LEFT
and RIGHT presses decrement or increment
sel_num with wrap around, allowing
continuous scrolling through all the three
images. For each sel_num value, the
corresponding bitmap is drawn using
GLCD_Bitmap(70, 50, 100, 125, (unsigned
char *)<IMAGE_pixel_data>);

This positions the character image near the
center of the screen. A small busy-wait
delay (image_delay(10)) is used to slow
down animation and avoid flicker. When the
joystick SELECT button is pressed, the loop
breaks and control returns to main_menu(),
effectively exiting the gallery.



Figure 3: First Image in the Gallery

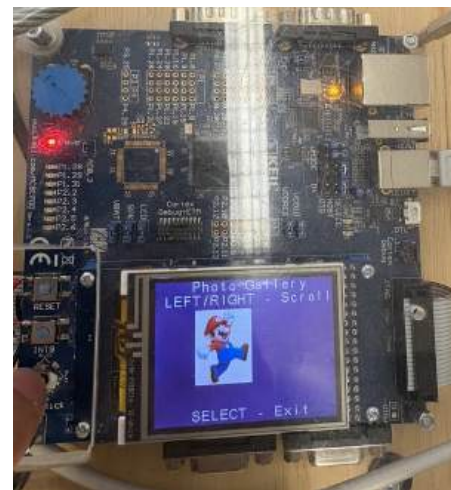


Figure 4: Second Image in the Gallery



Figure 5: *Third Image in the Gallery*

D. Audio Player Module

The audio player module is accessed from the main menu when the option is selected. The function `mp3_player()` is responsible for configuring the USB device stack so that the MCB1700 appears as an audio output device to the host PC, setting up Timer0 for the desired audio sampling rate, and configuring the DAC to output audio samples. The on-board potentiometer is periodically sampled using the ADC to implement a simple volume control, where lower ADC readings correspond to lower scaling of the audio sample written to the DAC.

On the user interface side, the audio player displays its header and a brief hint ("SELECT - exit") on the LCD. While audio is active, the joystick is monitored, pressing SELECT stops audio playback by disabling the timer and USB interrupts and returns control to the main menu. Note that the image displayed in the audio player, shown in Figure 6 was implemented using the same strategy (using GIMP) but this is just for extra aesthetics.

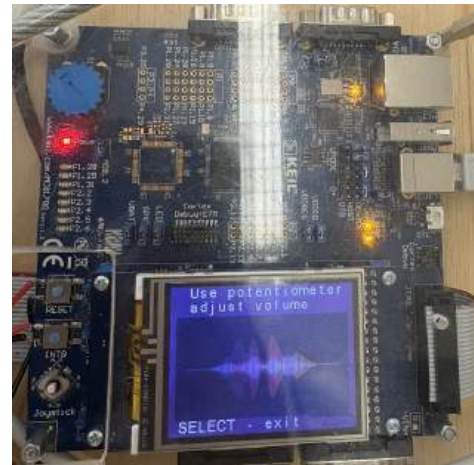


Figure 6: *The MP3 Audio Player*

E. Game Menu Design

The game menu is implemented in `game_menu.c` and provides a simple interface for selecting between the two custom games, Grid Hunter and Highway Dodge, or exiting back to the main menu. `restart_game_menu()` repeatedly calls `game_menu()` until the user chooses Exit, ensuring that players can launch multiple games in a row without navigating back through the main menu each time.

When `game_menu()` runs, it clears the LCD, sets a purple background, and prints the title "Game Menu" and instructions ("UP/DOWN - Scroll", "SELECT - Select"). It maintains a `sel_num` variable representing the current selection:

- If `sel_num == 0`, Grid Hunter (calls `grid_hunter()`, returns 0) is invoked.
- If `sel_num == 1`, Highway Dodge (calls `highway_dodge()`, returns 0) is invoked.

- If `sel_num == 2`, `Exit` (returns 1, `restart_game_menu()` breaks its loop, `returns to main_menu()` is invoked.

The currently selected option is drawn with a light gray background to highlight it. Joystick UP/DOWN moves through the three choices with bounds checking and wrap-around, and joystick SELECT activates the highlighted options similar to the original main menu.



Figure 7: The Internal Game Menu

F. Grid Hunter Game

Grid Hunter, implemented in `grid_hunter.c`, is a fast paced memory based game played on a 3x3 grid rendered in text characters on the LCD. The player must memorize the positions of briefly flashed “X” targets randomly on the grid and then move the cursor to select one of those positions.

The grid is drawn using horizontal dashed lines and vertical bars. Helper functions map logical rows and columns (0-2) onto specific LCD line and column numbers. `get_ln_pos()` returns LCD lines 2, 5, and 8 for rows 0, 1,

and 2. `get_col_pos()` returns columns 4, 9, and 16 for columns 0, 1, and 2.

Text based grid lines are drawn by `draw_rows()` and `draw_columns()`, and the player cursor is represented by a ‘*’ drawn at the current cell by `draw_cursor()`. Targets are displayed as ‘X’ characters in yellow using `draw_target()` and cleared with `erase_target()`.

The main function `play_grid_hunter()` controls the ten rounds of the game. In each round, the number of targets (`numTargets`) and their visible duration (`flashDiv`) depend on the current difficulty. Rounds 1 to 3 have 3 targets (easy), rounds 4 to 7 have 2 targets (medium), and lastly rounds 8 to 10 have 1 target (hard). For each target, a random (row, column) pair is generated using `rand() % 3`, with checks to ensure no duplicate positions. All targets for the round are drawn simultaneously, held visible for a short time via `cursor_delay(flashDiv)`, and then erased. The cursor is then shown at its current location (starting at the center cell). Inside an inner loop, the joystick controls movement, UP/DOWN/LEFT/RIGHT moves the cursor within the bounds of the 3x3 grid accordingly. SELECT evaluates the player’s guess, if the cursor position matches any of the target positions, the round is counted as a hit and the score is incremented. If the cursor position does not match any of the target positions, the round is counted as a miss and the score does not get incremented. Round and score information is printed at the bottom of the screen on each iteration. After ten rounds,

play_grid_hunter() returns the final score (0 to 10).

The grid_hunter_game_over() function displays a “GAME OVER” screen, showing the score and a two item menu: “Replay” and “Exit.” Joystick UP/DOWN toggles the highlighted option, and SELECT either restarts the game (by returning 0) or exits back to the game menu (by returning 1). The public function grid_hunter() wraps this logic in a loop that continues replaying the game until the player chooses Exit.



Figure 8: First Game, “Grid Hunter”

G. Highway Dodge Game

Highway Dodge, implemented in highway_dodge.c, is a lane changing game where the player controls a car that must dodge oncoming traffic. The road consists of three vertical lanes, and obstacle cars move from right to left toward the player.

The game defines a small Obstacle structure:

```
typedef struct {
```

```
    int row;  
    int col;  
    int active;  
} Obstacle;
```

An array obs[NUM_OBSTACLES] stores up to five simultaneously active obstacles. Each obstacle has a row (lane), a column (horizontal position), and an active flag. The three lane row positions are stored in lane_rows[NUM_LANES] = {2, 4, 6}, and the player’s car is always drawn in a fixed column PLAYER_COL = 2. The player car is drawn as a blue ‘A’ with draw_player(), and obstacles are drawn as red ‘#’ characters with draw_obstacle(). Separate erase functions overwrite old positions with spaces to maintain a clean display. Busy-wait delays via delay(div) control the game speed.

The core gameplay is contained in the game() function. The screen is cleared to white, the title of the game is drawn on line 0, and the score is initialized to 0 and printed on line 1. The player car starts in the middle lane. Joystick UP/DOWN changes lanes, pressing UP moves the car to a higher lane if available. Pressing DOWN moves to a lower lane if available. On each iteration of the main loop, with a probability of approximately 1 in 8 frames, a new obstacle is spawned at the far right column (19) in a randomly chosen lane, provided a free slot is available in the obstacle array. All active obstacles are moved one column left, if an obstacle moves past the left boundary (column < 0), it is deactivated and the score is incremented by one. The updated score is printed on the LCD. After movement,

collision detection checks whether any active obstacle shares both the lane row and column with the player car. If a collision is detected, the current score is returned and the game ends. The loop continues until a collision occurs, after which the final score is displayed on the GAME OVER screen which is the same one used for Grid Hunter. The wrapper `highway_dodge()` repeatedly calls `game()` and `game_over()` until the user selects Exit.



Figure 9: Second Game, “Highway Dodge”



Figure 10: The Game Over Screen

H. File Organization and Reusability

Each major module is separated into its own C file, with the game menu and main menu

logic in `Blinky.c` and `game_menu.c`, the Photo Gallery in `gallery.c`, and the two games in `grid_hunter.c` and `highway_dodge.c`. The image data arrays generated by GIMP are stored in dedicated C files and referenced via extern declarations. This organization keeps responsibilities clearly separated and makes it easy to add new games or gallery images by extending only the relevant modules and menu entries.

VI. EXPERIMENTAL RESULTS

Lab testing confirmed that most of the functionality designed behaved as expected on the physical MCB1700 hardware. The main menu responded reliably to joystick input, with the selected option highlighted correctly and wrapping between the first and last entries. Entering and exiting the photo gallery, audio player, and game menu operated according to the intended control flow, and each subapplication returned cleanly to the main menu without freezing or requiring a reset.

In the photo gallery, navigation with joystick LEFT and RIGHT successfully cycled through the three character images. Each image appeared with the correct orientation and aspect ratio, indicating that the GIMP conversion process and GLCD bitmap configuration were successful. The timing delay prevented flickering and SELECT consistently exited back to the main menu.

The game menu correctly handled selection of both Grid Hunter and Highway Dodge, and returned to the game menu after

each game session, or exited to the main menu when the Exit option was chosen. Grid Hunter's grid was clearly rendered on the LCD, with the cursor and targets updating smoothly. The number of visible targets changed as rounds progressed, demonstrating the difficulty adjustment logic. The game correctly identified hits and misses, updated the score out of 10, and displayed a coherent GAME OVER screen, from which the Replay and Exit options operated as intended and designed.

Highway Dodge also performed well. Lane changes triggered by joystick UP/DOWN were visually immediate, and obstacles spawned at random times and lanes, moving steadily from right to left. When obstacles passed completely off the screen, the score increased and was accurately reflected on the LCD. Collision detection correctly ended the game when an obstacle reached the same lane and column as the player car, and the GAME OVER screen displayed the final score along with functioning Replay and Exit options.

The main source of difficulty and frustration was the audio player module. While the USB audio pipeline was implemented according to the reference design, its behaviour on hardware was very inconsistent. In one test, audio from the host PC successfully played through the MCB1700 speaker, and the potentiometer adjusted the volume as expected. However, without changing any code, subsequent attempts produced no sound output even though the LCD continued to indicate that the MP3 player was active and the board

was recognized as an audio device by the PC. Disabling and re-enabling Timer() and the USB interface did not restore consistent playback, suggesting an issue within the USB audio driver, endpoint configuration, or host side audio routing. Due to time constraints, this problem could not be completely diagnosed and resolved. Aside from the intermittent audio issue, the remainder of the media center performed according to the project requirements. Menu navigation, image display, and both games operated reliably across multiple test sessions, demonstrating that the modular architecture and state based control design worked effectively on the embedded platform.

VII. CONCLUSIONS

The media center project successfully integrates multiple embedded software concepts into a single interactive application running on the Keil MCB1700 development board. A modular, menu driven architecture enabled clean separation between the main menu, photo gallery, audio player, game menu, and individual game modules. The photo gallery demonstrated the complete workflow of preparing custom images in GIMP, converting them into RGB565 C arrays, and rendering them on the LCD. The two original games, Grid Hunter and Highway Dodge, illustrated how relatively simple data structures and joystick driven control can provide engaging interactive experiences on a resource constrained embedded platform.

Although the audio player did not operate consistently, its partial success

confirmed that the underlying configuration of ADC, DAC, timer, and USB peripherals was largely correct. The remaining instability highlights the complexity of real-time USB audio streaming and suggests directions for future work, such as deeper debugging of the USB driver, improved buffer management, or experimenting with alternative sampling rates and endpoint settings. Addressing these issues would further strengthen the robustness of the system.

Beyond satisfying the project specifications, this work reinforced key embedded systems concepts such as modular design, state based control flow, interrupt handling, peripheral configuration, and user interface construction on basic embedded hardware. The experience of integrating multiple peripherals into a coherent application and diagnosing real-world problems such as unpredictable USB behaviour provides valuable preparation for larger embedded projects and professional practice in the future with embedded system design.

REFERENCES

[1] Lewis, Daniel Wesley. *Fundamentals of Embedded Software with the ARM Cortex-M3*. Pearson, 2013.

[2] G. N. Khan, “Embedded System CPUs: ARM7, Cortex M3”, p. 1-71, *Toronto Metropolitan University*, 2025, <https://www.ecb.torontomu.ca/%7Ecourses/coe718/lectures/Embedded-CPU-ARM7-M3.pdf>

[3] G. N. Khan, “ARM7, Cortex M3 Programming”, p. 1-43, *Toronto Metropolitan University*, 2025, <https://www.ecb.torontomu.ca/%7Ecourses/coe718/lectures/ARM7-M3-Programming.pdf>

[4] G. N. Khan, “RT Scheduling”, p. 1-60, *Toronto Metropolitan University*, 2025, <https://www.ecb.torontomu.ca/%7Ecourses/coe718/lectures/Multitasking-RTX.pdf>

[5] G. N. Khan, “Hardware-Software Co-Design System Partitioning”, p. 1-48, *Toronto Metropolitan University*, 2025, <https://www.ecb.torontomu.ca/%7Ecourses/coe718/lectures/HS-Codesign-Overview.pdf>

[6] G. N. Khan, “Final Project: Media Center”, p. 1-5, *Toronto Metropolitan University*, 2025, <https://www.ecb.torontomu.ca/%7Ecourses/coe718/labs/Media-Center.pdf>

APPENDIX

blinky.c

```
16 #include <LPC17xx.H>                      /* NXP LPC17xx definitions */
17 #include "string.h"
18 #include "GLCD.h"
19 #include "LED.h"
20 #include "KBD.h"
21
22 #define __FI      1                        /* Font index 16x24 */
23 #define __USE_LCD  0                      /* Uncomment to use the LCD */
24
25 /* External modules for the three main features */
26 extern int gallery(void);
27 extern void restart_game_menu(void);
28 extern int mp3_player (void);
29
30 /*----- Main Menu Screen -----*/
31 * Draws and handles the top-level Media Center menu.
32 * Returns when one of the subapplications finishes.
33 /*-----*/
34 int main_menu (void) {                    /* Main Program */
35     uint32_t joystick_motion;
36     int sel_num = 0;                      /* 0: Gallery, 1: Audio, 2: Games */
37
38     /* Initial main menu banner and instructions */
39     GLCD_Clear(Purple);                  /* Clear graphical LCD display */
40     GLCD_SetBackColor(Purple);
41     GLCD_SetTextColor(White);
42     GLCD_DisplayString(0, 0, 1, "||||Media Center||||");
43     GLCD_SetTextColor(White);
44     GLCD_DisplayString(1, 0, 1, "      Main Menu      ");
45     GLCD_DisplayString(2, 0, 1, "      UP/DOWN - Scroll  ");
46     GLCD_DisplayString(3, 0, 1, "      SELECT - Select   ");
47
48     while (1) {
49         /* Read joystick once per loop iteration */
50         joystick_motion = get_button();
51
52         /* Update selection index with UP/DOWN (3 entries: 0,1,2) */
53         if (sel_num<3 && sel_num>=0){
54             if (joystick_motion == KBD_DOWN){
55                 sel_num++;
56             }
57             else if (joystick_motion == KBD_UP){
58                 sel_num --;
59             }
60         }
61         else {
62             /* Out-of-range protection (wrap back to first item) */
63             sel_num = 0;
64         }
65     }
66 }
```

```

65
66 /* ----- Option 0: Photo Gallery ----- */
67 if (sel_num == 0){
68     GLCD_SetTextColor(White);
69     GLCD_SetBackColor(LightGrey); /* highlight */
70     GLCD_DisplayString(5, 0, __FI, "\1Photo gallery");
71     GLCD_SetBackColor(Purple);
72     GLCD_DisplayString(6, 0, __FI, "\2Audio");
73     GLCD_DisplayString(7, 0, __FI, "\3Game selection");
74
75     joystick_motion = get_button();
76     if (joystick_motion == KBD_SELECT){
77         GLCD_Clear(Purple);
78         gallery();
79         break;
80     }
81 }
82
83 /* ----- Option 1: Audio Player ----- */
84 else if (sel_num == 1) {
85     GLCD_SetTextColor(White);
86     GLCD_SetBackColor(Purple);
87     GLCD_DisplayString(5, 0, __FI, "\1Photo gallery");
88     GLCD_SetBackColor(LightGrey);
89     GLCD_DisplayString(6, 0, __FI, "\2Audio");
90     GLCD_SetBackColor(Purple);
91     GLCD_DisplayString(7, 0, __FI, "\3Game selection");
92
93     joystick_motion = get_button();
94     if (joystick_motion == KBD_SELECT){
95         GLCD_Clear(Purple);
96         mp3_player();
97         break;
98     }
99 }
100
101 /* ----- Option 2: Game Selection ----- */
102 else if (sel_num == 2){
103     GLCD_SetTextColor(White);
104     GLCD_SetBackColor(Purple);
105     GLCD_DisplayString(5, 0, __FI, "\1Photo gallery");
106     GLCD_DisplayString(6, 0, __FI, "\2Audio");
107     GLCD_SetBackColor(LightGrey);
108     GLCD_DisplayString(7, 0, __FI, "\3Game selection");
109
110     joystick_motion = get_button();
111     if (joystick_motion == KBD_SELECT){
112         GLCD_Clear(Purple);
113         restart_game_menu();
114         break;
115     }
116 }
117 }
118 }
119
120 /*----- main -----*/
121 * Hardware initialization + perpetual main menu loop.
122 *-----*/
123 int main(void){
124     LED_Init (); /* On-board LEDs (not heavily used) */
125     GLCD_Init(); /* LCD controller */
126     KBD_Init(); /* Joystick */
127
128     /* Always return to main menu after each subapplication finishes */
129     while(1){
130         main_menu();
131     }
132 }
133

```


gallery.c

```
16 #include <LPC17xx.H>                      /* NXP LPC17xx definitions */
17 #include "string.h"
18 #include "GLCD.h"
19 #include "LED.h"
20 #include "KBD.h"
21
22 /* Image pixel data arrays generated from GIMP (RGB565, 100x125) */
23 extern SUPER_MARIO_pixel_data[];
24 extern SONIC_pixel_data[];
25 extern STEVE_pixel_data[];
26
27
28 #define __FI      1                        /* Font index 16x24 */
29
30 /******
31  * Simple busy-wait delay used to slow down image updates on LCD
32  * Parameter: div - larger value -> shorter effective delay
33  *****/
34 int image_delay(int div){
35     int i;
36     int val = 10000000/div;
37     int count;
38     for (i = 0; i<val; i++){
39         count ++;
40     }
41 }
42
43 /*-----
44  Photo Gallery
45  Displays one of three character images and scrolls with LEFT / RIGHT.
46  -----*/
47 int gallery (void) {
48     uint32_t joystick_motion;
49     int sel_num=0;                        /* 0: Sonic, 1: Mario, 2: Steve */
50
51     /* Initial gallery header and instructions */
52     GLCD_Clear(Purple);
53     GLCD_SetBackColor(Purple);
54     GLCD_SetTextColor(White);
55     GLCD_DisplayString(0, 0, 1, "    Photo Gallery    ");
56     GLCD_DisplayString(1, 0, 1, "LEFT/RIGHT - Scroll ");
57     GLCD_DisplayString(9, 0, 1, "SELECT - Exit    ");
58 }
```

```

59 while(1){
60     joystick_motion = get_button();
61
62     /* Scroll right through images with wrap-around 0 -> 1 -> 2 -> 0 */
63     if (joystick_motion == KBD_RIGHT){
64         if (sel_num == 2){
65             sel_num = 0;
66         }
67         else{
68             sel_num++;
69         }
70     }
71     /* Scroll left through images with wrap-around 0 <- 1 <- 2 <- 0 */
72     else if (joystick_motion == KBD_LEFT){
73         if (sel_num == 0){
74             sel_num = 2;
75         }
76         else{
77             sel_num --;
78         }
79     }
80     /* Exit gallery on SELECT (return to main menu) */
81     else if (joystick_motion == KBD_SELECT){
82         break;
83     }
84
85     /* Draw the currently selected character image */
86     if (sel_num == 0){
87         GLCD_Bitmap ( 70, 50, 100, 125, (unsigned char *)SONIC_pixel_data);
88     }
89
90     else if (sel_num == 1){
91         GLCD_Bitmap ( 70, 50, 100, 125, (unsigned char *)SUPER_MARIO_pixel_data);
92     }
93
94     else if (sel_num == 2) {
95         GLCD_Bitmap ( 70, 50, 100, 125, (unsigned char *)STEVE_pixel_data);
96     }
97     image_delay(10);      /* Small delay to reduce flicker and slow down scrolling */
98 }
99
100 }
101

```

game_menu.c

```
1  #include <LPC17xx.H>                                /* NXP LPC17xx definitions */
2  #include "string.h"
3  #include "GLCD.h"
4  #include "LED.h"
5  #include "KBD.h"
6  #include <stdlib.h>
7
8  /* Prototypes for the two games */
9  extern int highway_dodge(void);
10 extern int grid_hunter(void);
11
12 /*-----
13  * game_menu
14  * - Draws the Game Menu and handles joystick navigation.
15  * - Returns 0 when a game finishes (stay in game menu loop),
16  *   and 1 when user selects "Exit" (return to main menu).
17  *-----*/
18
19 int game_menu(){
20     int joystick_motion;
21     int sel_num = 0;          /* 0: Grid Hunter, 1: Highway Dodge, 2: Exit */
22
23     /* Initial Game Menu banner and instructions */
24     GLCD_Clear(Purple);
25     GLCD_SetBackColor(Purple);
26     GLCD_SetTextColor(White);
27     GLCD_DisplayString(0, 0, 1, "      Game Menu      ");
28     GLCD_DisplayString(1, 0, 1, "  UP/DOWN - Scroll  ");
29     GLCD_DisplayString(2, 0, 1, "  SELECT - Select   ");
30
31     while (1) {
32         joystick_motion = get_button();
33
34         /* Update selection index with UP/DOWN (3 entries: 0,1,2) */
35         if (sel_num<3 && sel_num>=0){
36             if (joystick_motion == KBD_DOWN){
37                 sel_num++;
38             }
39             else if (joystick_motion == KBD_UP){
40                 sel_num --;
41             }
42         }
43         else {
44             /* Safety: reset if sel_num somehow goes out of range */
45             sel_num = 0;
46         }
47     }
```

```

48      /* ----- Option 0: Grid Hunter ----- */
49      if (sel_num == 0){
50          GLCD_SetTextColor(White);
51          GLCD_SetBackColor(LightGrey); /* highlight */
52          GLCD_DisplayString(4, 0, 1, "\1Grid Hunter");
53          GLCD_SetBackColor(Purple);
54          GLCD_DisplayString(5, 0, 1, "\2Highway Dodge");
55          GLCD_DisplayString(6, 0, 1, "\3Exit");
56          joystick_motion = get_button();
57          if (joystick_motion == KBD_SELECT){
58              grid_hunter(); /* play game */
59              return 0;      /* back to game menu */
60          }
61      }
62
63      /* ----- Option 1: Highway Dodge ----- */
64      else if (sel_num == 1) {
65          GLCD_SetTextColor(White);
66          GLCD_SetBackColor(Purple);
67          GLCD_DisplayString(4, 0, 1, "\1Grid Hunter");
68          GLCD_SetBackColor(LightGrey);
69          GLCD_DisplayString(5, 0, 1, "\2Highway Dodge");
70          GLCD_SetBackColor(Purple);
71          GLCD_DisplayString(6, 0, 1, "\3Exit");
72          joystick_motion = get_button();
73          if (joystick_motion == KBD_SELECT){
74              highway_dodge(); /* play game */
75              return 0;      /* back to game menu */
76          }
77      }
78
79      /* ----- Option 2: Exit back to Main Menu ----- */
80      else if (sel_num == 2){
81          GLCD_SetTextColor(White);
82          GLCD_SetBackColor(Purple);
83          GLCD_DisplayString(4, 0, 1, "\1Grid Hunter");
84          GLCD_DisplayString(5, 0, 1, "\2Highway Dodge");
85          GLCD_SetBackColor(LightGrey);
86          GLCD_DisplayString(6, 0, 1, "\3Exit");
87          joystick_motion = get_button();
88          if (joystick_motion == KBD_SELECT){
89              return 1;      /* request exit to main menu */
90          }
91      }
92  }
93 }
94

```

```

95  /*-----*/
96  * restart_game_menu
97  * - Wrapper that keeps reopening the Game Menu until the user
98  *   chooses "Exit" (game_menu returns 1).
99  *-----*/
100 void restart_game_menu() {
101     int exit_game_menu;
102     while(1){
103         exit_game_menu = game_menu();
104         if (exit_game_menu == 1){
105             break;    /* leave and go back to main menu */
106         }
107     }
108 }

```

grid_hunter.c

```

1  #include <LPC17xx.H>                /* NXP LPC17xx definitions */
2  #include "string.h"
3  #include "GLCD.h"
4  #include "LED.h"
5  #include "KBD.h"
6  #include <stdlib.h>
7  #include <stdio.h>
8
9  #define __FI 1                      /* Font index 16x24 */
10 #define __USE_LCD 0                 /* Uncomment to use the LCD */
11
12 /* ===== Helper delay used for cursor and simple timing ===== */
13 /* Busy-wait delay: bigger div -> shorter delay */
14 static void cursor_delay(int div){
15     int i;
16     int val = 10000000 / div; /* Choose loop count based on desired speed */
17     volatile int count = 0;
18     for (i = 0; i < val; i++){
19         count++;
20     }
21 }
22
23 /* ===== 3x3 grid ===== */
24
25 /* Map logical grid row [0..2] to LCD line number */
26 static int get_ln_pos(int ln){
27     int ln_num;
28     if (ln == 1){
29         ln_num = 5; /* Middle row of grid drawn on LCD line 5 */
30     }
31     else if (ln == 0){
32         ln_num = 2; /* Top row of grid drawn on LCD line 2 */
33     }
34     else { /* ln == 2 */
35         ln_num = 8; /* Bottom row of grid drawn on LCD line 8 */
36     }
37     return ln_num;
38 }
39

```

```

40  /* Map logical grid column [0..2] to LCD column number */
41  static int get_col_pos(int col){
42      int col_num;
43      if (col == 1){
44          col_num = 9; /* Middle column of grid drawn on LCD line 5 */
45      }
46      else if (col == 0){
47          col_num = 4; /* Top column of grid drawn on LCD line 5 */
48      }
49      else { /* col == 2 */
50          col_num = 16; /* Bottom column of grid drawn on LCD line 5 */
51      }
52      return col_num;
53  }
54
55  /* Draw the two horizontal grid lines */
56  static void draw_rows(void){
57      GLCD_SetTextColor(White);
58      GLCD_DisplayString(3,0,1, " ----- ");
59      GLCD_DisplayString(6,0,1, " ----- ");
60  }
61
62  /* Draw the two vertical grid lines */
63  static void draw_columns(void){
64      int i;
65      for (i = 1; i < 9 ; i++){
66          GLCD_DisplayChar(i,6,1, '|');
67      }
68      for (i = 1; i < 9 ; i++){
69          GLCD_DisplayChar(i,13,1, '|');
70      }
71  }
72
73  static void draw_grid(void){
74      draw_rows();
75      draw_columns();
76  }
77
78  /* Draw the selector cursor at a given cell */
79  static void draw_cursor(int ln_pos, int col_pos){
80      GLCD_DisplayChar(ln_pos, col_pos, 1, '*');
81  }
82
83  /* Erase cursor at given cell */
84  static void erase_cursor(int ln_pos, int col_pos){
85      GLCD_DisplayChar(ln_pos, col_pos, 1, ' ');
86  }
87

```



```

88  /* Draw / erase target at a given logical cell */
89  static void draw_target(int row, int col){
90      int ln = get_ln_pos(row);
91      int cn = get_col_pos(col);
92      GLCD_SetTextColor(Yellow);
93      GLCD_DisplayChar(ln, cn, 1, 'X');
94  }
95
96  static void erase_target(int row, int col){
97      int ln = get_ln_pos(row);
98      int cn = get_col_pos(col);
99      GLCD_DisplayChar(ln, cn, 1, ' ');
100 }
101
102 /* ===== Core Grid Hunter Memory Game ===== */
103 /*
104 Rules:
105 - Each round, 1-3 targets 'X' flash briefly on the 3x3 grid.
106   * Rounds 1-3: 3 X
107   * Rounds 4-7: 2 X's
108   * Rounds 8-10: 1 X
109 - X's then disappear.
110 - Move the cursor '*' with the joystick (UP/DOWN/LEFT/RIGHT).
111 - Press SELECT when your cursor is on ANY of the X positions.
112 - 10 rounds total; score goes up by 1 for each correct hit.
113 */
114
115 static int play_grid_hunter(void){
116     int cursor_row = 1, cursor_col = 1; /*current cursor position in coords */
117     int prev_ln, prev_col;               /* previous LCD line/col for cursor (for erasing) */
118     int ln, cn;                          /* current LCD line/col for cursor */
119     int rounds = 0;                       /* round counter */
120     int score = 0;
121     int js;
122     char buf[32];
123
124     /* arrays for up to 3 targets per round */
125     int tRow[3], tCol[3];
126     int numTargets; /* how many X's in this round */
127     int flashDiv; /* how long X's are visible */
128     int i, j;
129     int unique; /* flag used when picking unique target positions */
130     int hit; /* 1 if player selected any correct target, else 0 */
131
132     GLCD_Clear(Purple);
133     GLCD_SetBackColor(Purple);
134     GLCD_SetTextColor(White);
135     GLCD_DisplayString(0,0,1,"  GRID HUNTER ");
136     GLCD_DisplayString(1,0,1,"Memori ze X's & pick");
137

```

```

138     GLCD_SetBackColor(Black);
139     GLCD_ClearLn(2,1);
140     draw_grid();
141
142     /* initial cursor in centre cell */
143     cursor_row = 1;
144     cursor_col = 1;
145     ln = get_ln_pos(cursor_row);
146     cn = get_col_pos(cursor_col);
147     draw_cursor(ln, cn);
148
149     while (rounds < 10){
150
151         /* difficulty: number of X's and flash time */
152         if (rounds < 3){
153             numTargets = 3;          /* easy: 3 targets */
154             flashDiv   = 1;          /* longer show time */
155         }
156         else if (rounds < 7){
157             numTargets = 2;          /* medium: 2 targets */
158             flashDiv   = 2;          /* shorter time */
159         }
160         else {
161             numTargets = 1;          /* hard: 1 target */
162             flashDiv   = 3;          /* shortest time */
163         }
164
165         /* generate unique target cells */
166         for (i = 0; i < numTargets; i++){
167             unique = 0;
168             while (!unique){
169                 tRow[i] = rand() % 3;
170                 tCol[i] = rand() % 3;
171                 unique = 1;
172                 for (j = 0; j < i; j++){
173                     if (tRow[i] == tRow[j] && tCol[i] == tCol[j]){
174                         unique = 0;
175                         break;
176                     }
177                 }
178             }
179         }
180
181         /* hide cursor while flashing targets */
182         erase_cursor(ln, cn);
183

```

```

184      /* flash all X's briefly */
185      for (i = 0; i < numTargets; i++){
186          draw_target(tRow[i], tCol[i]);
187      }
188
189      cursor_delay(flashDiv); /* brief visible time */
190
191      for (i = 0; i < numTargets; i++){
192          erase_target(tRow[i], tCol[i]);
193      }
194
195      /* redraw cursor at current position after targets disappear */
196      ln = get_ln_pos(cursor_row);
197      cn = get_col_pos(cursor_col);
198      draw_cursor(ln, cn);
199
200      /* show round and score info on bottom lines */
201      GLCD_SetBackColor(Purple);
202      GLCD_SetTextColor(White);
203      sprintf(buf, "Round: %d/10", rounds + 1);
204      GLCD_DisplayString(9, 0, 1, buf);
205      sprintf(buf, "Score: %d", score);
206      GLCD_DisplayString(8, 0, 1, buf);
207
208      /* inner loop: wait until user presses SELECT */
209      while (1){
210          js = get_button();
211
212          prev_ln = ln;
213          prev_col = cn;
214
215          if (js == KBD_UP && cursor_row > 0){
216              cursor_row--;
217          }
218          else if (js == KBD_DOWN && cursor_row < 2){
219              cursor_row++;
220          }
221          else if (js == KBD_LEFT && cursor_col > 0){
222              cursor_col--;
223          }
224          else if (js == KBD_RIGHT && cursor_col < 2){
225              cursor_col++;
226          }
227          else if (js == KBD_SELECT){
228              /* evaluate this round: hit if cursor matches ANY X */
229              hit = 0;
230              for (i = 0; i < numTargets; i++){
231                  if (cursor_row == tRow[i] && cursor_col == tCol[i]){
232                      hit = 1;
233                      break;
234                  }
235              }

```

```

236
237     if (hit){
238         score++;
239         GLCD_SetBackColor(Purple);
240         GLCD_SetTextColor(Green);
241         GLCD_DisplayString(2,0,1,"          HIT!          ");
242     } else {
243         GLCD_SetBackColor(Purple);
244         GLCD_SetTextColor(Red);
245         GLCD_DisplayString(2,0,1,"          MISS          ");
246     }
247     cursor_delay(5);
248     GLCD_SetBackColor(Purple);
249     GLCD_ClearLn(2,1);
250     break; /* go to next round */
251 }
252
253 /* update cursor drawing if moved */
254 ln = get_ln_pos(cursor_row);
255 cn = get_col_pos(cursor_col);
256 if (ln != prev_ln || cn != prev_col){
257     erase_cursor(prev_ln, prev_col);
258     draw_cursor(ln, cn);
259 }
260
261     cursor_delay(2);
262 }
263
264     rounds++;
265 }
266
267     return score;
268 }
269
270 /* ===== Game over screen and replay handling ===== */
271
272 static int grid_hunter_game_over(int score){
273     int js_sel;
274     int sel_num = 0;
275     char buf[32];
276
277     GLCD_Clear(Purple);
278     GLCD_SetBackColor(Purple);
279     GLCD_SetTextColor(White);
280     GLCD_DisplayString(0,0,1,"          GAME OVER          ");
281     sprintf(buf, "          SCORE: %2d/10          ", score);
282     GLCD_DisplayString(1,0,1, buf);
283     GLCD_DisplayString(2,0,1,"          UP/DOWN - Scroll          ");
284     GLCD_DisplayString(3,0,1,"          SELECT - Select          ");
285

```

```

286     /* initial menu state */
287     GLCD_SetBackColor(LightGrey);
288     GLCD_DisplayString(5,0,1,"Replay");
289     GLCD_SetBackColor(Purple);
290     GLCD_DisplayString(6,0,1,"Exit");
291
292     while (1){
293         js_sel = get_button();
294
295         if (sel_num == 0 || sel_num == 1){
296             if (js_sel == KBD_DOWN){
297                 sel_num = 1; /* move selection to exit */
298             } else if (js_sel == KBD_UP){
299                 sel_num = 0; /* move selection to replay */
300             }
301         } else {
302             sel_num = 0;
303         }
304
305         if (sel_num == 0){
306             GLCD_SetBackColor(LightGrey);
307             GLCD_DisplayString(5,0,1,"Replay");
308             GLCD_SetBackColor(Purple);
309             GLCD_DisplayString(6,0,1,"Exit");
310             if (js_sel == KBD_SELECT){
311                 return 0; /* replay */
312             }
313         } else {
314             GLCD_SetBackColor(Purple);
315             GLCD_DisplayString(5,0,1,"Replay");
316             GLCD_SetBackColor(LightGrey);
317             GLCD_DisplayString(6,0,1,"Exit");
318             if (js_sel == KBD_SELECT){
319                 return 1; /* exit to game menu */
320             }
321         }
322     }
323 }
324
325 /* Public entry point used by game_menu.c . */
326 int grid_hunter(void){
327     int score;
328     int exit_flag;
329
330     while (1){
331         score = play_grid_hunter();
332         exit_flag = grid_hunter_game_over(score);
333
334         if (exit_flag){
335             /* return to game menu */
336             return 0;
337         }
338         /* otherwise loop and replay */
339     }
340 }

```

highway_dodge.c

```
1  #include <LPC17xx.H>                /* NXP LPC17xx definitions */
2  #include "string.h"
3  #include "GLCD.h"
4  #include "LED.h"
5  #include "KBD.h"
6  #include <stdlib.h>
7  #include <stdio.h>
8
9  #define __FI      1                  /* Font index 16x24 */
10 #define __USE_LCD  0                  /* Uncomment to use the LCD */
11
12 #define NUM_LANES   3
13 #define NUM_OBSTACLES 5
14 #define PLAYER_COL  2
15
16 typedef struct {
17     int row;
18     int col;
19     int active;
20 } Obstacle;
21
22 /* Busy-wait delay used to slow down the game loop / animations */
23 void delay(int div) {
24     int i;
25     int val = 10000000 / div; /* Larger div => smaller val => shorter delay */
26     volatile int count = 0; /* volatile prevents compiler from optimizing it away */
27     for (i = 0; i < val; i++) { /* Simple loop to burn CPU cycles */
28         count++;
29     }
30 }
31
32 /* Draw the player car at a given LCD row in the fixed player column */
33 static void draw_player(int row) {
34     GLCD_SetBackColor(White);
35     GLCD_SetTextColor(Blue);
36     GLCD_DisplayChar(row, PLAYER_COL, 1, 'A'); /* player car 'A' */
37 }
38
39 /* Erase the player car by printing a space in its position */
40 static void erase_player(int row) {
41     GLCD_SetBackColor(White); /* Background remains white */
42     GLCD_DisplayChar(row, PLAYER_COL, 1, ' '); /* Overwrite with space character */
43 }
44
45 /* Draw a single obstacle car if it's active */
46 static void draw_obstacle(const Obstacle *o) {
47     if (!o->active) return; /* If obstacle slot is not in use, nothing to draw */
48     GLCD_SetBackColor(White); /* same white background as road */
49     GLCD_SetTextColor(Red); /* Obstacles drawn in red */
50     GLCD_DisplayChar(o->row, o->col, 1, '#'); /* draw '#' to represent obstacle car */
51 }
52
```

```

53 /* Erase an obstacle by printing a space where it used to be */
54 static void erase_obstacle(const Obstacle *o) {
55     if (!o->active) return; /* Ignore inactive obstacles */
56     GLCD_SetBackColor(White); /* Background remains white */
57     GLCD_DisplayChar(o->row, o->col, 1, ' '); /* Overwrite with space character */
58 }
59
60 /* "Highway Dodge" lane-change game.
61 Move your car up/down with the joystick to avoid oncoming cars. */
62 int game(void) {
63     const int lane_rows[NUM_LANES] = { 2, 4, 6 }; /* three lanes on the road */
64     Obstacle obs[NUM_OBSTACLES];
65     int i;
66     int lane = 1; /* start in middle lane */
67     int score = 0; /* player's score increments for each obstacle that passes safely */
68     int js; /* joystick input value from get_button() */
69     char score_str[20]; /* buffer to store formatted score string */
70
71     /* initialise obstacle list */
72     for (i = 0; i < NUM_OBSTACLES; i++) {
73         obs[i].active = 0;
74         obs[i].row = 0;
75         obs[i].col = 0;
76     }
77
78     GLCD_Clear(White);
79     GLCD_SetBackColor(White);
80     GLCD_SetTextColor(Blue);
81     GLCD_DisplayString(0, 0, 1, " Highway Dodge ");
82
83     GLCD_SetTextColor(Black);
84     sprintf(score_str, "Score: 0 ");
85     GLCD_DisplayString(1, 0, 1, score_str);
86
87     draw_player(lane_rows[lane]);
88     delay(5);
89
90     /* Main game loop: runs until collision occurs and function returns score */
91     while (1) {
92         js = get_button();
93
94         /* lane change with joystick */
95         if (js == KBD_UP && lane > 0) {
96             erase_player(lane_rows[lane]);
97             lane--;
98             draw_player(lane_rows[lane]);
99         } else if (js == KBD_DOWN && lane < (NUM_LANES - 1)) {
100             erase_player(lane_rows[lane]);
101             lane++;
102             draw_player(lane_rows[lane]);
103         }

```



```

104
105 /* maybe spawn a new obstacle at the far right */
106 if ((rand() % 8) == 0) { /* ~1 in 8 frames */
107     for (i = 0; i < NUM_OBSTACLES; i++) { /* find a free obstacle slot */
108         if (!obs[i].active) { /* if this slot is inactive */
109             obs[i].active = 1; /* mark obstacle as active */
110             obs[i].col = 19; /* stat at right-most column */
111             obs[i].row = lane_rows[rand() % NUM_LANES]; /* Random lane row for this car */
112             draw_obstacle(&obs[i]); /* draw new obstacle on screen */
113             break; /* stop after using one free slot */
114         }
115     }
116 }
117
118 /* move all active obstacles one step to the left */
119 for (i = 0; i < NUM_OBSTACLES; i++) {
120     if (obs[i].active) { /* only move currently active obstacles */
121         erase_obstacle(&obs[i]); /* erase obstacle at old position */
122         obs[i].col--; /* move obstacle one column left */
123
124         if (obs[i].col < 0) { /* if obstacle moved off left edge */
125             /* went off-screen safely -> score point */
126             obs[i].active = 0; /* free this obstacle slot */
127             score++; /* increase player's score */
128             GLCD_SetBackColor(White);
129             GLCD_SetTextColor(Black);
130             sprintf(score_str, "Score: %4d", score); /* update score string */
131             GLCD_DisplayString(1, 0, 1, score_str);
132         } else {
133             draw_obstacle(&obs[i]);
134         }
135     }
136 }
137
138 /* collision check: same lane and column as player */
139 for (i = 0; i < NUM_OBSTACLES; i++) {
140     if (obs[i].active && /* obstacle exists */
141         obs[i].row == lane_rows[lane] && /* obstacle row matches player's lane */
142         obs[i].col == PLAYER_COL) { /* obstacle column matches PLAYER_COL */
143         return score; /* collision, end game and return score */
144     }
145 }
146
147 delay(12);
148 }
149 }
150

```

```

151  /* Game-over screen and replay/exit logic*/
152  int game_over(int score){
153      int js_sel;
154      int sel_num = 0;
155      char score_str[20];
156      sprintf(score_str, "          SCORE: %d          ", score);
157      GLCD_Clear(Purple);
158      GLCD_SetBackColor(Purple);
159      GLCD_SetTextColor(White);
160      GLCD_DisplayString(0,0,1,"          GAME OVER          ");
161      GLCD_DisplayString(1,0,1,score_str);
162      GLCD_DisplayString(2, 0, 1, "          UP/DOWN - Scroll  ");
163      GLCD_DisplayString(3, 0, 1, "          SELECT - Select  ");
164      while(1){
165          js_sel = get_button();
166          if (sel_num == 0 || sel_num == 1){
167              if (js_sel == KBD_DOWN){
168                  sel_num++; /* move selection to exit */
169              }
170              else if (js_sel == KBD_UP){
171                  sel_num--; /* move selection to replay */
172              }
173          }
174          else {
175              sel_num = 0;
176          }
177
178          if (sel_num == 0){
179              GLCD_SetBackColor(LightGrey);
180              GLCD_DisplayString(5,0,1,"Replay");
181              GLCD_SetBackColor(Purple);
182              GLCD_DisplayString(6,0,1,"Exit");
183              js_sel = get_button();
184              if (js_sel == KBD_SELECT){
185                  return 0; /* replay */
186              }
187          }
188          else if (sel_num == 1){
189              GLCD_SetBackColor(Purple);
190              GLCD_DisplayString(5,0,1,"Replay");
191              GLCD_SetBackColor(LightGrey);
192              GLCD_DisplayString(6,0,1,"Exit");
193              js_sel = get_button();
194              if (js_sel == KBD_SELECT){
195                  return 1; /* exit to game menu */
196              }
197          }
198      }
199  }
200
201  /* Public entry point used by game_menu.c .  */
202  void highway_dodge(void){
203      int game_score = 0;
204      int reset;
205
206      while(1){
207          game_score = game();
208          reset = game_over(game_score);
209
210          if (reset == 1)
211              break;
212      }
213  }

```